

Tensor Network Simulation of Grover’s Algorithm: Overcoming the Statevector Memory Wall

Veselin Petrov Nikolaev
University of Luxembourg
Faculty of Science, Technology and Medicine
Luxembourg, Luxembourg
veselin.nikolaev.001@student.uni.lu

Abstract

Our prior Bachelor Semester Project (BSP2) [1] showed that full statevector simulation of Grover’s algorithm runs out of GPU memory at around $n = 31$ – 32 qubits. Here we ask whether tensor networks push that wall back. The implementation runs on quimb, cotengra, and kahypar on CPU rather than the NVIDIA cuTensorNet backend that was originally proposed, mainly for practical reasons discussed in Section 1.

We look at three things: the memory a tensor-network contraction is estimated to need, the bond dimension an MPS needs to represent the state exactly, and how far the memory estimate is from what actually gets used during a real contraction.

Short version: tensor networks did not help. Across $n = 2$ – 11 we found no sustained memory advantage over the statevector; the minimum bond dimension for exact MPS convergence climbed from 4 at $n = 5$ to 32 at $n = 11$; and measured memory ran well above the simple width-based estimate at every qubit count tested, which says something about how much that estimate should be trusted on its own.

Grover’s circuit, at least at these sizes and with this implementation, is not a good candidate for tensor-network compression. That is a fairly narrow claim, but the reasons behind it – entanglement growth and gate-count blowup from transpilation – are the interesting part.

1 Introduction

A full statevector for an n -qubit system needs 2^n complex amplitudes stored in memory, and that requirement doubles with every extra qubit. Our earlier BSP2 GPU experiments [1] hit this wall at around $n = 31$ – 32 qubits. The obvious next question is whether a different representation of the same state can push that ceiling further out.

Tensor networks are one candidate. Rather than materializing all 2^n amplitudes, the state (or the circuit acting on it) is factored into a network of smaller tensors, and how much this actually saves depends entirely on the structure of what’s being represented – if the intermediate tensors or MPS bond dimensions stay small, the savings can be large; if not, they aren’t. A useful mental picture is a matrix product state (MPS): a chain of small tensors linked by bond indices, where the bond dimension controls how expressive that chain can be [2].

Grover’s algorithm is a reasonable test case precisely because it’s a mixed bag: the oracle and diffusion operator both have clean, structured circuits, but the algorithm’s whole point is to spread interference across the entire search space. So rather than assuming tensor networks would win here, the goal was just to measure it.

The proposal originally called for NVIDIA cuTensorNet on GPU. In practice, the project ended up on the CPU-based quimb + cotengra + kahypar stack instead – mostly because these libraries made it much easier to actually inspect contraction paths and run the MPS experiments in the development environment available. That switch matters for how the results should be read: this is a comparison between simulation *methods*, not a GPU-vs-CPU hardware benchmark. Section 3.1 introduces these libraries in the context of related work.

This report is built around three research questions: (1) is there a qubit count at which tensor networks become more memory-efficient than a statevector? (2) how does the minimum bond dimension needed for exact convergence – a proxy for entanglement – scale with n ? and (3) where does the tested tensor-network approach stop being practically feasible, and why?

Section 2 covers background on Grover’s algorithm, tensor networks, and bond dimension; Section 3 the related work, libraries, and implementation; Section 4 the results; Section 5 the discussion; Section 6 wraps up.

2 Theoretical Background

2.1 Grover’s Algorithm

We assume familiarity with Grover’s algorithm [3] from prior work. Briefly: given a Boolean oracle marking exactly one target state $|x^*\rangle$ in a search space of $N = 2^n$ states, Grover’s algorithm uses repeated oracle-diffusion cycles to amplify the target’s amplitude. After $k_{\text{opt}} = \lfloor \frac{\pi}{4} \sqrt{N} \rfloor$ iterations, measurement yields $|x^*\rangle$ with high probability. The algorithm is implemented as n Hadamard gates (initialization), followed by k_{opt} iterations of (1) a phase oracle and (2) a diffusion operator.

2.2 Tensor Networks and Contraction

A quantum circuit can be represented as a tensor network: each quantum gate is a tensor (a multi-dimensional array), and evaluating the circuit means contracting the network (summing over shared indices). A single-qubit gate is a 2×2 matrix (a rank-2 tensor); a two-qubit gate is $2 \times 2 \times 2 \times 2$ (rank-4).

An alternative to full contraction is to approximate the state as a Matrix Product State (MPS), where the state is decomposed into a chain of small tensors connected by bonds, each with a bond dimension χ controlling both compression and accuracy; Section 2.3 defines χ precisely.

Contraction path. The order in which pairs of tensors are contracted determines both the memory cost (the size of intermediate tensors created during contraction) and the time cost (the number

of scalar multiplications). Finding the truly optimal contraction path is NP-hard in general; cotengra instead runs a bounded, randomized search over candidate orderings and returns the best one found within a time/repeat budget. As discussed in Section 3.3, this search is stochastic and can return different-quality orderings across repeated runs on the same circuit, particularly for larger, more heavily decomposed circuits.

Contraction width (W). We define

$$W = \log_2 \left(\max_s |T_s| \right)$$

where $|T_s|$ is the size of the intermediate tensor created at contraction step s . W measures peak memory during contraction, expressed in qubits' worth of dimension, directly comparable to the statevector's $\log_2(2^n) = n$. This is the quantity used throughout the results in Section 4 to compare tensor-network memory against the statevector.

2.3 Bond Dimension and Entanglement

Bond dimension definition. If the state is partitioned into left and right halves via singular value decomposition (SVD), the bond dimension χ is the number of singular values kept. Keeping all singular values gives an exact representation; truncating to the χ largest is a lossy compression. The minimum χ for which truncation loses no information (to numerical precision) is what we measure and report.

Connection to entanglement. For a bipartition of a quantum state, the Schmidt rank is related to the MPS bond dimension. In particular, the entanglement entropy is bounded by $\log_2 \chi$. A product state has $\chi = 1$, while an arbitrary n -qubit state may require a bond dimension as large as $2^{\lfloor n/2 \rfloor}$ for an appropriate bipartition [2]. Therefore, increasing bond dimension provides a practical indication that the state is becoming harder to compress as an MPS.

2.4 Why Tensor Networks Can Fail for General Circuits

Two mechanisms can defeat tensor-network compression for a given circuit: (1) genuine entanglement growth, where the required bond dimension approaches $2^{n/2}$ regardless of contraction strategy, and (2) contraction path cost, where even a circuit with bounded entanglement can still be expensive to contract if a good ordering is hard to find or if gate decomposition inflates the network's structure. Section 4 presents evidence for both mechanisms in this project's data.

3 Related Work and Implementation

3.1 Related Work and Libraries

Tensor network simulation. Markov and Shi [4] established that contracting a tensor network computes quantum circuit amplitudes, and that circuits with bounded treewidth admit efficient (polynomial-time) contraction.

Contraction heuristics. Finding optimal contraction paths is NP-hard in general. cotengra [5] uses randomized, hyperparameter-tuned heuristics, optionally including hypergraph-partitioning-based methods via kahypar [6], to find good (not necessarily optimal) paths without exhaustive search.

Quantum simulation libraries. quimb [7] is a widely-used Python library for tensor-network quantum simulation, used here both for exact contraction (via its cotengra integration) and for approximate MPS simulation with explicit bond-dimension truncation.

Grover studies. Prior experimental work on Grover's algorithm has focused on hardware demonstrations at small qubit counts or classical statevector simulation (such as our prior BSP2 baseline [1]). This project extends that line of work with a systematic tensor-network-based study.

The project uses three main libraries:

- (1) **quimb** (QUantum Information Many-Body): A Python library for tensor network operations, including quantum circuit evaluation via tensor contraction and via approximate matrix product state (MPS) simulation with a truncation parameter.
- (2) **cotengra** (Contraction Engine): A library that searches for good contraction orderings for tensor networks using randomized, hyperparameter-tuned heuristics (finding the truly optimal ordering is NP-hard in general).
- (3) **kahypar**: A hypergraph partitioner that cotengra can optionally use internally as one of its candidate methods for finding contraction orderings, alongside a simpler greedy method.

3.2 Circuit Construction and Transpilation

Grover's algorithm is constructed in Qiskit (reusing the oracle and diffuser construction from our prior BSP2 codebase [1]) and converted for use with quimb via the qiskit-quimb bridge. This bridge only recognizes elementary single- and two-qubit gates; Grover's oracle and diffuser use an arbitrary-arity multi-controlled-X gate (mcx), which Qiskit's simulator handles natively but the bridge does not. The circuit is therefore transpiled to a basis of rz, sx, x, and cx gates before conversion, which recursively decomposes each mcx into a chain of smaller gates. This decomposition is treated as a genuine part of the cost being measured, not a workaround to be discounted: it is the same gate-count blowup any concrete backend would face when asked to execute an n -controlled gate using only elementary two-qubit interactions.

3.3 Search Stability of the Contraction Optimizer

cotengra's randomized search (HyperOptimizer) can produce varying contraction width estimates across repeated runs on the same circuit, particularly for $n \geq 9$. To mitigate this variance, the search was run multiple independent times per qubit count, keeping the minimum width found (three repeats for $n \leq 9$; a single run for $n = 10, 11$ due to the search taking tens of minutes to hours). Repeated multi-trial search was used by default throughout, as single-shot greedy search proved unstable and occasionally returned implausibly poor orderings for larger circuits.

3.4 Bond Dimension Measurement via MPS Truncation

For each qubit count n and a set of randomly chosen target states, we run the Grover circuit through quimb’s MPS circuit simulator at a range of fixed maximum bond dimensions $\chi \in \{2, 4, 6, 8, 10, 12, 16, 20, 24, 28, 32, 48, 64\}$, and separately with χ uncapped as an exact reference. For each χ , we compare the resulting amplitude for the target bitstring against the uncapped reference amplitude, and record the smallest χ at which the error drops to (and stays at) zero to numerical precision. This is repeated for three independent random target states per n ; we report both the worst-case (maximum) and best-case (minimum) threshold across those three targets, since different target states can require different bond dimensions.

We note that this MPS-based approach is conceptually distinct from the exact-contraction approach used for W and C above: MPS truncation is lossy by construction (bond dimension is an approximation parameter), while the contraction-width approach is always exact. The two therefore answer different aspects of the same underlying research questions and are reported as separate, complementary results rather than one extending the other.

3.5 Real Memory Measurement

For small n (up to $n = 9$), we additionally perform the tensor-network contraction for real (not just estimating its cost) and measure peak memory using Python’s tracemalloc module, which tracks memory allocated through Python’s memory allocator (this captures NumPy array allocations on standard builds, but can undercount memory allocated outside the Python allocator, such as some BLAS working buffers). To avoid wasting compute on contractions that would be infeasible to actually run, we only attempt real execution when the rehearsed width estimate implies a feasible memory footprint (below roughly 2 GB); the $n = 10, 11$ points in the main sweep, whose estimated widths implied far larger footprints, were excluded from this real-execution comparison for that reason.

3.6 Experimental Setup

To evaluate the tensor network representations systematically, experiments were structured as follows: For each qubit count n , the same Grover circuit was constructed, transpiled, and converted into a tensor network. The contraction optimizer was then used to find a contraction path. From this path, the maximum intermediate tensor size (width W) was recorded. Separately, the identical circuit was simulated as an MPS with different maximum bond dimensions.

The experiments measured scaling from $n = 2$ up to $n = 11$. Three random target states were tested for the bond-dimension experiments to capture variation, while other structural tests relied on a single representative target or randomized contraction paths. Real execution and memory tracking were conducted up to $n = 9$. Estimated paths were pushed to $n = 11$, beyond which the contraction optimizer itself became too computationally expensive to run reliably within the project’s constraints.

3.7 Experiment Suite

The project consists of four main experiments, summarized in Table 1:

- (1) **TN vs. statevector memory** ($n = 2\text{--}11$): Compares theoretical statevector memory ($16 \cdot 2^n$ bytes) against the tensor network’s rehearsed memory estimate ($16 \cdot 2^W$ bytes). Answers research question 1. (Figure 7)
- (2) **Bond dimension growth** ($n = 5\text{--}11$, three random targets per n): Tracks the minimum χ needed for exact MPS convergence, worst- and best-case. Answers research question 2. (Figure 8)
- (3) **Bond dimension convergence detail** (largest n tested, one target): Shows reconstruction error vs. χ for a single representative case, illustrating the mechanism behind Figure 8’s growth curve. (Figure 9)
- (4) **Rehearsed vs. real memory** ($n = 2\text{--}9$): Compares cotengra’s width-based estimate against actually measured peak memory from real contraction. Informs research question 3 and validates the estimate itself. (Figure 10)

4 Results

4.1 Tensor Network vs. Statevector Memory (Figure 7)

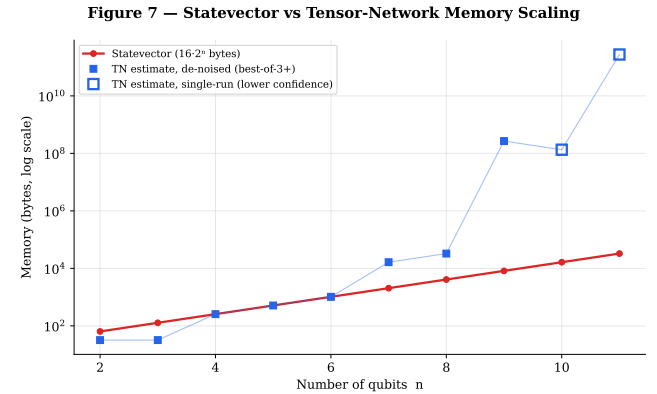


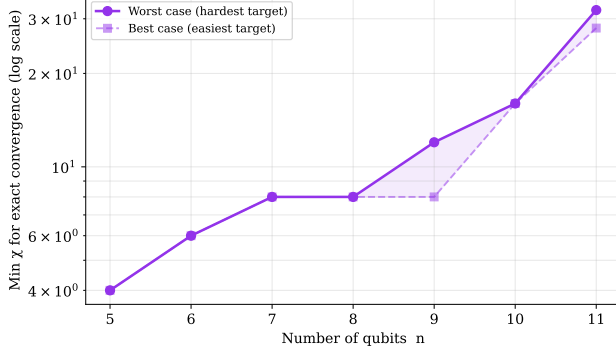
Figure 7: Statevector memory vs. tensor-network rehearsed memory estimate, $n = 2\text{--}11$. Hollow markers at $n = 10, 11$ are single-run, lower-confidence estimates (Section 3.3).

Figure 7 plots statevector memory ($16 \cdot 2^n$ bytes) against the tensor network’s rehearsed estimate ($16 \cdot 2^W$ bytes) on a log scale, $n = 2\text{--}11$. Filled markers are de-noised over multiple independent search repeats (Section 3.3); the hollow ones at $n = 10, 11$ are single runs and should only be read as order-of-magnitude.

Below $n \approx 6$ both curves are small and close together, so there’s no real advantage either way. Past $n = 7$ the TN estimate starts climbing fast, and by $n = 10\text{--}11$ – where the contraction optimizer itself is already getting hard to run – it sits roughly six to seven orders of magnitude *above* the statevector curve. So for RQ1: no, tensor networks don’t become more memory-efficient than the statevector anywhere in this range. The gap goes the wrong way, and it widens.

Table 1: Summary of Experiments

Experiment	Qubit range	Repetitions	Main measurement
TN vs. statevector	2–11	Multiple optimizer runs	Estimated memory
Bond dimension	5–11	3 targets	Minimum exact χ
Convergence	11	1 target	Error vs. χ
Real memory	2–9	Multiple runs	Peak memory

Figure 8 — Minimum Bond Dimension for Exact Convergence vs n Figure 8: Minimum bond dimension χ for exact MPS convergence vs. n , worst- and best-case across 3 random targets per n (shaded band shows the spread).

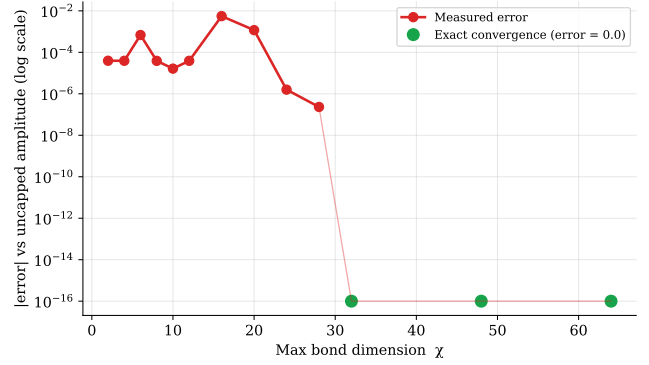
4.2 Minimum Bond Dimension (Figure 8)

Table 2 and Figure 8 show the minimum χ needed for exact MPS convergence, measured across three random target states per n for $n = 5$ –11. The shaded band in the figure is just the spread between the easiest and hardest of those three targets.

Table 2: Minimum bond dimension for exact convergence (worst case across 3 random targets)

n	$\chi_{\text{worst-case}}$
5	4
6	6
7	8
8	8
9	12
10	16
11	32

Bond dimension climbs steadily with n , and the jump from $n = 10$ to $n = 11$ is the sharpest one – it doubles, from 16 to 32. That answers RQ2: whatever entanglement Grover’s oracle and diffuser are generating, it grows with qubit count, and the MPS needs a correspondingly larger bond dimension to stay exact.

Figure 9 — Convergence vs Bond Dimension ($n=11$)Figure 9: Reconstruction error vs. bond dimension χ for the largest n tested (one representative target). Green points converged exactly and are plotted at an arbitrary floor purely so the log scale renders.

4.3 Bond Dimension Convergence Detail (Figure 9)

Figure 9 shows reconstruction error (log scale) versus bond dimension χ for the largest n tested, for one representative target state. The error does not decay smoothly with increasing χ ; instead it remains large (and is not strictly monotonic – occasional non-monotonic bumps reflect that each gate’s MPS truncation is a locally, not globally, optimal decision) until a fairly sharp transition region, after which it drops to numerical zero. This illustrates the mechanism behind Figure 8: convergence for this circuit behaves more like a threshold effect than a gradual improvement, so a coarse χ grid can misreport the true threshold if it only samples powers of two.

4.4 Rehearsed vs. Real Memory (Figure 10)

Figure 10 compares cotengra’s width-based estimate ($16 \cdot 2^W$ bytes) against actual peak memory measured with tracemalloc, $n = 2$ –9.

Measured memory beats the estimate at every point – by about three orders of magnitude at small n , closing to one or two orders of magnitude by $n = 9$. That’s not really surprising once you think about what W is: it only tracks the size of the single largest intermediate tensor at one contraction step, not everything else sitting in memory at the same time (the other coexisting tensors, plus NumPy/quimb/Python overhead). Practically, this means the $16 \cdot 2^W$ formula is a lower bound, not a usable prediction of how much memory a run will actually need.

Figure 10 — Rehearsed Estimate vs Actually Measured Memory

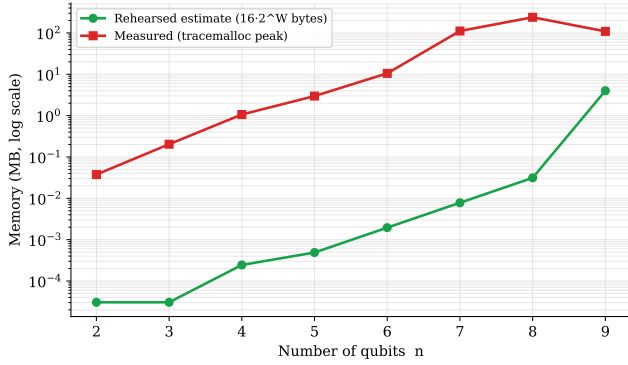


Figure 10: Rehearsed width-based memory estimate ($16 \cdot 2^W$ bytes) vs. actually measured peak memory (tracemalloc), $n = 2-9$.

5 Analysis and Discussion

5.1 Why Tensor Networks Struggle for Grover’s Algorithm

Two things in the data above seem to be working against tensor-network compression here.

Representation complexity grows with n . Figure 8’s bond dimension curve is basically the state getting harder to compress as n increases – more correlations across the bipartitions the MPS has to cut through. This doesn’t pin down exactly which entanglement mechanism is responsible, but the trend itself is clear enough.

Transpilation inflates the gate count. Section 3.2 already covers this: turning `mcx` into elementary gates for tensor-network compatibility adds a lot of extra structure to the network, which makes the contraction-path search harder before bond dimension even enters the picture.

Neither of these is a knock against tensor networks as a general idea – they just happen to both work against Grover’s circuit specifically. A circuit with a flatter entanglement profile and fewer decomposed gates could look completely different.

5.2 Limitations

Several methodological limitations restrict the generalizability of these findings:

Hardware: The tensor-network experiments were CPU-based, while our prior BSP2 statevector result [1] used GPU hardware. Thus, the results demonstrate that this specific CPU tensor-network implementation did not outperform the previous GPU statevector implementation, rather than definitively proving tensor networks are universally worse than statevectors on equivalent hardware.

Small qubit range: The tensor-network experiments only reached $n = 11$. This restricted scale does not allow for definitive claims about the asymptotic behavior of arbitrarily large Grover circuits.

Random target states: The bond-dimension experiment utilized three random target states per n . While reasonable for an initial empirical estimate, three samples are insufficient to fully

characterize the behavior of every possible target state in the search space.

Contraction optimizer randomness: As noted in Section 3.3, the heuristic search for contraction paths introduces variance, meaning the recorded memory estimates reflect the best path found within the search budget rather than the guaranteed theoretical minimum.

Memory measurement: The use of `tracemalloc` captures memory allocated through Python’s allocator, but does not capture external memory such as certain low-level C or BLAS working buffers, rendering it a lower bound on the true memory footprint.

5.3 Practical Ceiling Reached in This Project

Real contraction stopped being feasible past $n = 9$, and even the single-search width estimates only made it to $n = 11$ before a single contraction attempt was taking tens of minutes to hours. That ceiling is well short of our earlier $n = 31-32$ qubit GPU baseline [1], but it’s not a separate problem – it’s the same entanglement growth and optimizer instability from Sections 4.2 and 3.3 catching up with the project’s time budget.

6 Conclusion

This project built and evaluated a tensor-network simulation of Grover’s algorithm on `quimb`, `cotengra`, and `kahypar`, measuring estimated memory, bond dimension, and real contraction behavior across $n = 2-11$.

- (1) **Where does the memory crossover happen?** Nowhere in this range – by $n = 11$ the tensor-network estimate is six to seven orders of magnitude *above* the statevector, and the gap keeps widening.
- (2) **How does bond dimension scale?** From $\chi = 4$ at $n = 5$ up to $\chi = 32$ at $n = 11$, with the sharpest jump right at the end, between $n = 10$ and $n = 11$.
- (3) **Where does it become intractable?** Real contraction died out past $n = 9$; single-run estimates held on until $n = 11$. Figure 9 points to why: convergence in χ is a threshold effect, not a gradual one, which makes the whole thing harder to scale predictably.

None of this means tensor networks are a bad idea in general – it means they were a bad fit for *this* circuit, at these sizes, with this implementation. Their advantage only shows up when the circuit’s structure keeps the intermediate tensors and bond dimensions small, and Grover’s algorithm doesn’t do that.

Several natural extensions could sharpen this comparison: repeating the experiments with NVIDIA `cuTensorNet` on GPU, as originally proposed, to see whether GPU acceleration shifts the crossover point; testing circuits more naturally suited to tensor networks (shallow, low-entanglement, or 1D circuits) to check whether these scaling difficulties are specific to Grover’s circuit or more general; and, if a larger time budget or better hardware were available, pushing the contraction-optimizer search and the qubit range further to see whether the trends observed here continue to hold.

References

- [1] V. P. Nikolaev. *Classical Simulation and Experimental Analysis of Grover's Algorithm*. Bachelor Semester Project 2 (BSP2) Report, Faculty of Science, Technology and Medicine, University of Luxembourg, 2026. https://github.com/veselinnikolaev/Grover-s-Algorithm/blob/main/report/pdf/English_Report.pdf.
- [2] Tensor Network. *Matrix Product State / Tensor Train*. TensorNetwork.org, accessed August 2026. <https://tensornetwork.org/mps/>.
- [3] Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.
- [4] I. L. Markov and Y. Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.
- [5] J. Gray. cotengra: Contraction Path Finding for Tensor Networks. <https://github.com/jcmgray/cotengra>, 2023.
- [6] S. Schlag et al. KaHyPar: A Multilevel Hypergraph Partitioner. *ACM Transactions on Mathematical Software*, 46(2):17, 2020.
- [7] J. Gray. quimb: A Python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.